

ASP.NET Core Middleware & The Request Pipeline

Comprehensive Guide to Architecture, Implementation, and Trade-offs

1. What is Middleware and the Request Pipeline?

The Concept

In ASP.NET Core, **Middleware** refers to software components assembled into an application pipeline to handle requests and responses. Each component chooses whether to pass the request on to the next component in the pipeline and can perform operations before and after the next component is invoked.

Why is it Needed?

Middleware solves the problem of **cross-cutting concerns**. Logic such as authentication, logging, CORS, global exception handling, and routing applies to almost all HTTP requests. Embedding this logic directly into Controllers or Services violates the Single Responsibility Principle and causes massive code duplication. Middleware extracts this logic into a unified, reusable pipeline.

The Architecture (The "Russian Doll" Model)

The pipeline operates on a bidirectional flow. When a request arrives, it passes through each middleware sequentially. Once a middleware generates a response (or an endpoint is reached), the response travels back out through the same middleware in reverse order.

```
Client Request
    ↓
[ Middleware 1 (e.g., Exception Handler) ]
    ↓ invokes next()
  [ Middleware 2 (e.g., Routing) ]
    ↓ invokes next()
[ Middleware 3 (e.g., Authentication) ]
    ↓ invokes next()
[ Terminal Endpoint (e.g., Controller) ]
    ↓ returns response
  [ Middleware 3 (Post-processing) ]
    ↓ returns response
  [ Middleware 2 (Post-processing) ]
    ↓ returns response
```

```
[ Middleware 1 (Post-processing) ]
```



```
Client Response
```

2. Core Principles and Mechanics

- **RequestDelegate:** A middleware is essentially a function that receives the `HttpContext` and a delegate pointing to the `next` middleware in the chain.
- **Short-Circuiting:** A middleware does not have to call `next()`. If a request is invalid, or if it matches a static file, the middleware can write the response and return immediately, preventing subsequent middleware from executing.
- **Order is Critical:** The order in which middleware components are added in `Program.cs` dictates their execution order. A misconfigured order leads to severe security or functional bugs (e.g., placing Authorization before Authentication).

Standard Pipeline Order

While customizable, a robust production API typically registers middleware in this precise sequence:

1. **Exception Handling:** Catches exceptions from anywhere later in the pipeline.
2. **HSTS / HTTPS Redirection:** Enforces secure transport immediately.
3. **Static Files:** Serves files early to avoid unnecessary processing (short-circuits if a file is found).
4. **Routing:** Determines which endpoint the request maps to (but doesn't execute it yet).
5. **CORS:** Evaluates Cross-Origin constraints.
6. **Authentication:** Identifies the user based on tokens/cookies.
7. **Authorization:** Verifies the user has permission to access the resolved route.
8. **Endpoints:** Executes the actual Controller or Minimal API logic.

3. Types of Middleware Registration

Type 1: Inline (Anonymous) Middleware using `app.Use()`

You can define middleware directly in `Program.cs`. This is suitable for very brief, single-purpose logic.

```
app.Use(async (context, next) =>
{
    // Pre-processing
    context.Response.Headers.Append("X-Custom-Header", "ASP.NET Core");

    // Call the next middleware
    await next(context);

    // Post-processing (executes on the way back out)
});
```

Trade-off: Highly readable for small logic, but rapidly degrades maintainability if complex business logic is placed inline.

Type 2: Terminal Middleware using `app.Run()`

`app.Run()` explicitly terminates the pipeline. It does not receive a `next` parameter.

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Pipeline terminated here.");
});
```

4. Implementing Custom Middleware Classes

For production code, middleware should be encapsulated into distinct classes. There are two ways to write middleware classes in ASP.NET Core.

Approach A: Conventional Middleware

This is the traditional approach. The class is instantiated as a **Singleton** once during application startup.

```

public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestTimingMiddleware> _logger;

    // Injected at startup (Singleton scope only)
    public RequestTimingMiddleware(RequestDelegate next, ILogger<RequestTimingMiddleware>
logger)
    {
        _next = next;
        _logger = logger;
    }

    // Invoked on every request. Scoped services MUST be injected here, not in the
constructor.
    public async Task InvokeAsync(HttpContext context, IScopedBusinessService
scopedService)
    {
        var start = DateTime.UtcNow;

        await _next(context); // Continue pipeline

        var duration = DateTime.UtcNow - start;
        _logger.LogInformation($"Request {context.Request.Path} took
{duration.TotalMilliseconds}ms");
    }
}

// Extension method for clean registration
public static class RequestTimingMiddlewareExtensions
{
    public static IApplicationBuilder UseRequestTiming(this IApplicationBuilder builder)
    {
        return builder.UseMiddleware<RequestTimingMiddleware>();
    }
}

```

Critical Dependency Injection Technical Flaw: Because conventional middleware is instantiated as a Singleton, injecting a Scoped service (like Entity Framework's `DbContext`) into the constructor will cause an exception. The scoped service would be captured in a singleton context, causing severe concurrency issues. **Scoped services must always be injected into the `InvokeAsync` method signature.**

Approach B: Factory-Based Middleware (IMiddleware)

Introduced to solve the Singleton DI confusion, factory-based middleware implements the `IMiddleware` interface. These are resolved from the Dependency Injection container on a per-request basis, meaning they behave as **Scoped** dependencies.

```
public class TenantResolutionMiddleware : IMiddleware
{
    private readonly ITenantService _tenantService; // Can safely inject scoped services here

    public TenantResolutionMiddleware(ITenantService tenantService)
    {
        _tenantService = tenantService;
    }

    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        var tenantId = context.Request.Headers["X-Tenant-ID"].FirstOrDefault();
        _tenantService.SetCurrentTenant(tenantId);

        await next(context);
    }
}
```

Registration Difference: Factory-based middleware must explicitly be registered in the DI container before being added to the pipeline.

```
// In Program.cs
builder.Services.AddTransient<TenantResolutionMiddleware>(); // 1. Register in DI
var app = builder.Build();
app.UseMiddleware<TenantResolutionMiddleware>(); // 2. Add to pipeline
```

5. Technical Notes and Best Practices

- **Never block threads:** Middleware is in the hot path of every HTTP request. Always use asynchronous methods (`await next(context)`) and never use `.Wait()` or `.Result` , as this will lead to thread pool starvation.
- **Avoid modifying the response body after next():** Once `next()` is called and the downstream components start writing to the response stream, the headers are sent to the client. Attempting to modify headers or status codes after `next()` returns will throw a `System.InvalidOperationException: Headers are read-only, response has already started` .
- **Use MapWhen for branching:** If middleware should only apply to certain paths (e.g., only requests starting with `/api/admin`), use `app.MapWhen()` to branch the pipeline rather than putting if-statements inside the middleware itself.